ReWear

מסמך אפיון לפרויקט הגמר: אתר למכירה וקנייה של בגדי יד שנייה, עם תשלום באשראי וצבירת נקודות למוכרות.

טכנולוגיות	צוות	עמודים	הגשה
Node · Express · MongoDB	5 מפתחות	10	19.10

הגנה	גרסה
20.10	2.0

0 איך משתמשים במסמך הזה

זה החוזה של הצוות. כל מה שכתוב כאן סגור ולא נפתח מחדש באמצע העבודה — שמות שדות, שמות routes, מבנה תשובות. אם מישהי צריכה לשנות משהו, משנים כאן קודם ומודיעים לכולן.

הכשל הנפוץ ביותר בפרויקט של חמישה אנשים הוא לא באג, אלא שתי בנות שהניחו הנחות שונות. אחת מחזירה `{ user }` והשנייה מצפה ל־`{ data }`. שעה של סיכום עכשיו חוסכת שלושה ימים בהמשך.

הסעיפים החשובים ביותר לקריאה ראשונה: סעיף 3 (איך התשלום עובד), סעיף 12 (מדריך התחלה), סעיף 13 (מי עושה מה) וסעיף 14 (מי יכולה לעבוד לבד ומי מחכה).

1 מה המערכת עושה

משתמשת נרשמת ומעלה בגדים למכירה. משתמשות אחרות מחפשות, מסננות לפי מגדר, קטגוריה, סוג פריט, מידה ומצב, שומרות למועדפים, ורוכשות בכרטיס אשראי. המוכרת מקבלת נקודות בשווי המכירה, שאותן היא יכולה להוציא על פריטים אחרים באתר. מנהלת רואה את כל התמונה ומנהלת את המערכת.

שלוש רמות ההרשאה

פעולה	אורחת	משתמשת	מנהלת
צפייה בחנות, חיפוש, סינון ומיון	✓	✓	✓
עמוד מוצר, אודותינו, איך זה עובד, צרו קשר	✓	✓	✓

פעולה	אורחת	משתמשת	מנהלת
שליחת הודעה בטופס צרו קשר	✓	✓	✓
פרסום מוצר	—	✓	✓
עריכה ומחיקה של מוצר	—	רק שלה	כל מוצר
מועדפים	—	✓	✓
רכישה, הזמנות ונקודות	—	✓	✓
ניהול משתמשות, מוצרים והודעות	—	—	✓
Dashboard וסטטיסטיקות	—	—	✓

חשוב מאוד להגנה. ההבדל בין המשתמשות חייב להיות אכוף בשרת, לא רק בהסתרת כפתורים. המרצה יכולה לפתוח Postman, לשלוח בקשה ל־ `/api/admin/users` עם טוקן של משתמשת רגילה, ולצפות לקבל 403.

---

2 החלטות טכנולוגיה — סגורות

נושא	ההחלטה	למה
שרת	Node.js + Express	דרישת הקורס
בסיס נתונים	MongoDB Atlas + Mongoose	Atlas לענן, Compass להדגמה בהגנה
סיסמאות	`bcryptjs`, 10 סבבים	אסור לשמור סיסמה גלויה, גם לא בפרויקט לימודי
הזדהות	JWT בתוקף 7 ימים	פשוט יותר מ־sessions, לא דורש אחסון בשרת
איפה נשמר הטוקן	`localStorage` בשם `rewear_token`	אותו שם בכל הקבצים, בלי גרסאות שונות
תמונות	Cloudinary, העלאה מהדפדפן	ראו סעיף 10
Frontend	HTML + CSS + Vanilla JS, `fetch`	בלי React. הדרישה היא הפרדה לשלושה סוגי קבצים
גרפים	Chart.js דרך CDN	שתי שורות קוד לגרף
שפת ממשק	עברית, RTL	להחליט פעם אחת ולא לערבב

מבנה תשובות אחיד מהשרת

כל endpoint מחזיר את המידע ישירות, בלי עטיפה. שגיאה מחזירה תמיד אובייקט עם שדה `message` בעברית:

```
200 { "_id": "...", "title": "שמלה פרחונית", ... } 400 [ { ... } ,{ ... } ] 200 { "message": "חסר שדה חובה: מחיר" } 401 { "message": "נדרשת התחברות" } 402 { "message": "אין מספיק נקודות" } 403 { "message": "אין לך הרשאה לפעולה הזו" } 404 { "message": "המוצר לא נמצא" } 409 { "message": "המוצר כבר נמכר" }
```

בצד הלקוח כותבות פונקציה אחת ב־`common.js` שכולן קוראות לה, כדי שהטיפול בטוקן ובשגיאות יהיה במקום אחד:

```
async function api(path, options = {}) { const token = localStorage.getItem('rewear_token'); const res = await fetch(path, { ...options, headers: { 'Content-Type': 'application/json', ...(token && { Authorization: `Bearer ${token}` }), ...options.headers } }); const data = await res.json().catch(() => ({})); if (!res.ok) throw new Error(data.message || 'משהו השתבש'); return data; }
```

---

3 איך התשלום והנקודות עובדים

הקונה משלמת בכרטיס אשראי. המוכרת מקבלת נקודות בשווי המכירה. הנקודות משמשות לקנייה באתר. אין עמלות, אין משיכת כסף.

העיקרון

זו כלכלה סגורה: כסף נכנס לאתר, ויוצא ממנו רק בצורת פריטים. בגלל זה המערכת לא צריכה להעביר כסף אמיתי לאף אחת, וזו בדיוק הסיבה שהמודל הזה עובד בפרויקט לימודי בלי סליקה אמיתית. נקודה אחת שווה שקל אחד — בלי המרות ובלי חישובים.

מי	מה קורה	איפה זה נשמר
קונה	משלמת באשראי או בנקודות שצברה	`orders.paymentMethod`
מוכרת	מקבלת נקודות בשווי המחיר	`users.points` עולה
מוצר	עובר ל־`sold` ויוצא מהחנות	`products.status`

שתי דרכי תשלום בעמוד אחד

בעמוד התשלום הקונה בוחרת:

- **כרטיס אשראי** — תמיד זמין. הטופס נבדק בצד הלקוח והכרטיס לא עוזב את הדפדפן.
- **נקודות** — מופיע רק אם יש לה מספיק. אם לא, האפשרות מוצגת מנוטרלת עם הסבר כמה חסר.

מה קורה עם פרטי האשראי

> **מספר הכרטיס לא נשלח לשרת ולא נשמר בשום מקום.** הוא נבדק ב־JavaScript בדפדפן ואז נזרק. לשרת נשלחות רק ארבע הספרות האחרונות, לצורך תצוגה בהזמנה.
>
> זה קריטי. אם המרצה תראה מספר כרטיס שמור במונגו, זו לא הערה קטנה — זו הפרה של תקן PCI שאסורה גם בפרויקט לימודי. התשובה "לא שמרנו, בדקנו בצד לקוח בלבד" היא תשובה מקצועית.

בראש עמוד התשלום מופיעה שורה: *מצב הדגמה — לא מתבצע חיוב אמיתי.* זה לא מחליש אתכן, זה מראה שידעתן מה אתן עושות.

אימות הכרטיס — אלגוריתם Luhn

זו הבדיקה שכל חברות האשראי בעולם משתמשות בה, והיא עשר שורות. תבדקו: `4242424242424242` יעבור, `1234567812345678` ייפסל.

```
function isValidCard(number) { const digits = number.replace(/\D/g, ''); if (digits.length < 13 || digits.length > 19) return false; let sum = 0, double = false; for (let i = digits.length - 1; i >= 0; i--) { let d = Number(digits[i]); if (double) { d *= 2; if (d > 9) d -= 9; } sum += d; double = !double; } return sum % 10 === 0; }
```

בנוסף בודקים: שם בעל הכרטיס לא ריק, תוקף בפורמט `MM/YY` ועדיין לא פג, ו־CVV בן שלוש ספרות.

הרכישה בשרת

שלוש פעולות שחייבות לקרות יחד. שימו לב לתנאים *בתוך* השאילתות — הם מה שמונע כפילויות:

```
// 1. תופסים את המוצר רק אם הוא עדיין זמין — בפעולה אטומית אחת const product = await Product.findOneAndUpdate( { _id: req.body.productId, status: 'available' }, { status: 'sold' }, { new: true } ); if (!product) return res.status(409).json({ message: 'המוצר כבר נמכר' }); // 2. אי אפשר לקנות מוצר של עצמך if (product.sellerId.toString() === req.user.id) { await Product.findByIdAndUpdate(product._id, { status: 'available' }); return res.status(400).json({ message: 'אי אפשר לקנות מוצר של עצמך' }); } // 3. תשלום בנקודות — מחייבים רק אם יש מספיק, גם כאן בפעולה אחת if (req.body.paymentMethod === 'points') { const buyer = await User.findOneAndUpdate( { _id: req.user.id, points: { $gte: product.price } }, { $inc: { points: -product.price } }, { new: true } ); if (!buyer) { await Product.findByIdAndUpdate(product._id, { status: 'available' }); return res.status(402).json({ message: 'אין מספיק נקודות' }); } } // 4. המוכרת מקבלת נקודות בשווי המכירה await User.findByIdAndUpdate(product.sellerId, { $inc: { points: product.price } }); // 5. יוצרים את ההזמנה const order = await Order.create({ buyerId: req.user.id, sellerId: product.sellerId, productId: product._id, price: product.price, pointsEarned: product.price, paymentMethod: req.body.paymentMethod, cardLast4: req.body.cardLast4 || null, shippingCity: req.body.shippingCity, status: 'completed' });
```

> **שווה להזכיר בהגנה.** אם ישאלו למה לא בדקתן זמינות עם `findById` ואז עדכנתן — יש לכן תשובה אמיתית: בין הבדיקה לעדכון קונה אחרת יכולה להספיק לקנות. `findOneAndUpdate` עושה את שניהם בפעולה אחת. אותו רעיון בדיוק בבדיקת `points: { $gte: price }`, שמונעת מקונה עם 50 נקודות לפתוח שני טאבים ולקנות שני פריטים ב־40 בו־זמנית.

---

4 מבנה בסיס הנתונים

חמישה Collections. השמות באנגלית ברבים ובאותיות קטנות, כמו שמונגו יוצר אותם.

users

```
{ _id: ObjectId, name: String, // חובה email: String, // חובה, ייחודי, נשמר lowercase password: String, // חובה, hash של bcrypt, select: false role: String, // "user" | "admin", ברירת מחדל "user" points: Number, // ברירת מחדל 0, min 0 city: String, phone: String, avatar: String, // URL, אופציונלי createdAt: Date }
```

products

```
{ _id: ObjectId, title: String, // חובה description: String, price: Number, // חובה, min 0 brand: String, // "Zara", "ללא מותג" gender: String, // women | men | kids category: String, // clothing | shoes | accessories type: String, // dresses | sneakers | bags ... (5 סעיף) size: String, condition: String, // new-tags | new | excellent | very-good | good color: String, images: [String], // מערך URLs, לפחות אחד sellerId: ObjectId, // ref: 'User' city: String, status: String, // "available" | "sold", ברירת מחדל "available" views: Number, // 0 ברירת מחדל createdAt: Date }
```

orders

```
{ _id: ObjectId, buyerId: ObjectId, // ref: 'User' sellerId: ObjectId, // ref: 'User' productId: ObjectId, // ref: 'Product' price: Number, // מועתק מהמוצר ברגע הקנייה pointsEarned: Number, // מה שהמוכרת קיבלה paymentMethod: String, // "card" | "points" cardLast4: String, // ארבע ספרות בלבד, או null shippingCity: String, status: String, // "completed" | "cancelled" createdAt: Date }
```

המחיר מועתק לתוך ההזמנה ולא נשלף מהמוצר. אם המוכרת תערוך את המחיר אחר כך, ההזמנה הישנה תישאר נכונה.

favorites

```
{ _id: ObjectId, userId: ObjectId, // ref: 'User' productId: ObjectId, // ref: 'Product' createdAt: Date }
favoriteSchema.index({ userId: 1, productId: 1 }, { unique: true });
```

messages

הודעות מטופס "צרו קשר". המנהלת רואה אותן ומסמנת כטופלו.

```
{ _id: ObjectId, name: String, email: String, subject: String, body: String, isRead: Boolean, // ברירת מחדל false createdAt: Date }
```

---

5 עץ הקטגוריות והמידות

שלוש רמות סינון: **מגדר** → **קטגוריה** → **סוג פריט**. זה בדיוק המבנה של אתרי יד שנייה אמיתיים, והוא נותן לכן סינון עשיר בלי לסבך את המסד — שלושה שדות מחרוזת פשוטים במוצר.

> **קובץ אחד לכל האתר.** העץ נשמר ב־ `public/js/categories.js` ונטען בכל עמוד שצריך אותו: טופס פרסום מוצר, סרגל הסינון בחנות, והתפריט הראשי. אף אחת לא מעתיקה את הרשימה פעמיים, וכשמוסיפים קטגוריה — מוסיפים במקום אחד.

```
const CATEGORIES = { women: { label: 'נשים', categories: { clothing: { label: 'בגדים', types: { dresses: 'שמלות', skirts: 'חצאיות', tops: 'טופים וחולצות', pants: 'מכנסיים', knitwear: 'סוודרים וקרדיגנים', jackets: 'מעילים וזקטים', sets: 'סטים', swimwear: 'בגדי ים', sportswear: 'בגדי ספורט', other: 'אחר' }}, shoes: { label: 'נעליים', types: { heels: 'נעלי עקב', flats: 'נעליים שטוחות', sneakers: 'סניקרס', boots: 'מגפיים', sandals: 'סנדלים' }}, accessories: { label: 'אביזרים', types: { bags: 'תיקים', jewelry: 'תכשיטים', belts: 'חגורות', hair: 'אביזרי שיער', glasses: 'משקפיים' }} }}, men: { label: 'גברים', categories: { clothing: { label: 'בגדים', types: { shirts: 'חולצות וטי שירט', pants: 'מכנסיים', suits: 'חליפות ובלייזרים', knitwear: 'סוודרים וקרדיגנים', jackets: 'זקטים ומעילים', swimwear: 'בגדי ים', other: 'אחר' }}, shoes: { label: 'נעליים', types: { sneakers: 'סניקרס', formal: 'נעליים אלגנטיות', boots: 'מגפיים', sandals: 'סנדלים' }}, accessories: { label: 'אביזרים', types: { belts: 'חגורות', hats: 'כובעים', bags: 'תיקים', glasses: 'משקפיים' }} }}, kids: { label: 'ילדים', categories: { clothing: { label: 'בגדים', types: { girls: 'בגדי בנות', boys: 'בגדי בנים', baby: 'תינוקות', costumes: 'תחפושות' }}, shoes: { label: 'נעליים', types: { girlsShoes: 'נעלי בנות', boysShoes: 'נעלי בנים' }}, accessories: { label: 'אביזרים', types: { bags: 'תיקים', hats: 'כובעים', other: 'אחר' }} }} };
```

מידות

רשימת המידות בטופס משתנה לפי המגדר והקטגוריה שנבחרו. זו לוגיקה של חמש שורות ב־JavaScript, והיא מונעת מצב שמישהי בוחרת מידה 42 לתיק.

```
const SIZES = { clothing: ['XS/34','S/36','M/38','L/40','XL/42','XXL/44','One Size'], shoes: ['36','37','38','39','40','41','42','43','44','45'], accessories: ['One Size'], kidsClothing: ['2 שנים','4 שנים','6 שנים','8 שנים','10 שנים','12 שנים','14 שנים'], kidsShoes: ['24','26','28','30','32','34','36','38'] }; const CONDITIONS = { 'new-tags': 'חדש עם תווית', 'new': 'חדש', 'excellent': 'מעולה', 'very-good': 'טוב מאוד', 'good': 'טוב' };
```

---

6 העמודים

עשרה עמודי HTML. הדרישה היא שלושה לפחות.

קובץ	מה יש בו	מי רואה
`index.html`	עמוד בית: באנר, חיפוש, שלושה כפתורי מגדר, תשעה פריטים אחרונים, שלוש נקודות מכירה	כולן
`shop.html`	**כל המוצרים בעמוד אחד.** סרגל סינון מלא בצד, מיון, חיפוש, דפדוף	כולן
`product.html`	עמוד מוצר לפי `?id=`: גלריית תמונות, כל הפרטים, מחיר בשקל ובדולר, פרטי המוכרת, לב מועדפים, כפתור רכישה	כולן
`login.html`	שני טאבים: התחברות והרשמה. אחרי הצלחה חוזר לעמוד שממנו הגיעה	אורחת
`checkout.html`	עמוד תשלום לפי `?id=`: סיכום הפריט, בחירת אשראי או נקודות, טופס כרטיס, עיר למשלוח	משתמשת
`profile.html`	חמישה טאבים: הפרטים שלי · הנקודות שלי · המוצרים שלי (כולל פרסום ועריכה) · מועדפים · ההזמנות והמכירות שלי	משתמשת
`how-it-works.html`	איך זה עובד: שלושה שלבים למוכרת, שלושה לקונה, הסבר על הנקודות, שאלות נפוצות	כולן

קובץ	מה יש בו	מי רואה
`about.html`	אודותינו: הסיפור, החזון של אופנה מעגלית, הצוות	כולן
`contact.html`	צרו קשר: טופס ששומר ל־`messages`, פרטי קשר, מפה	כולן
`admin.html`	ארבעה כרטיסי מספרים, שני גרפים, טבלת משתמשות, טבלת מוצרים, תיבת הודעות	מנהלת

ה־navbar וה־footer מגיעים מ־JavaScript. אל תעתיקו את התפריט לעשרה קבצים. שימו `<div id="navbar"></div>` ו־`<div id="footer"></div>` בכל עמוד, וב־`common.js` פונקציה שממלאת אותם לפי מי מחוברת. עשרה עמודים בלי זה יהפכו לסיוט תחזוקה תוך שבוע.

---

7 כל ה־Routes

זו הטבלה שכל אחת מסתכלת בה כשהיא כותבת `fetch`.

אימות — `/api/auth`

Method	נתיב	גוף הבקשה	מחזיר	הרשאה
POST	`/register`	name, email, password, city, phone	token, user	—
POST	`/login`	email, password	token, user	—
GET	`/me`	—	המשתמשת המחוברת כולל נקודות	משתמשת
PUT	`/me`	name, city, phone, avatar	המשתמשת המעודכנת	משתמשת

`password` לא חוזר אף פעם, בשום תשובה.

מוצרים — `/api/products`

Method	נתיב	פרמטרים / גוף	מחזיר	הרשאה
GET	`/`	search, gender, category, type, size, condition, brand, city, minPrice, maxPrice, sort, page	products, total, pages	—
GET	`/my`	—	המוצרים של המשתמשת	משתמשת

Method	נתיב	פרמטרים / גוף	מחזיר	הרשאה
GET	`/brands`	—	רשימת מותגים לסינון	—
GET	`/:id`	—	מוצר יחיד + פרטי מוכרת, מעלה `views`	—
POST	`/`	כל שדות המוצר	המוצר שנוצר	משתמשת
PUT	`/:id`	אותם שדות	המוצר המעודכן	בעלים או מנהלת
DELETE	`/:id`	—	הודעת הצלחה	בעלים או מנהלת

> **מלכודת שתעלה לכן שעה.** ב־Express הסדר קובע. אם `/:id` רשום לפני `/my`, אז `/api/products/my` ייכנס ל־route של מוצר בודד עם `id = "my"` ומונגו יזרוק שגיאת ObjectId. רשמו תמיד נתיבים קבועים לפני נתיבים עם פרמטר.

הזמנות — `/api/orders`

Method	נתיב	גוף הבקשה	מחזיר	הרשאה
POST	`/`	productId, paymentMethod, cardLast4, shippingCity	ההזמנה שנוצרה + היתרה החדשה	משתמשת
GET	`/my`	—	מה שקניתי	משתמשת
GET	`/sales`	—	מה שמכרתי, כולל נקודות שהתקבלו	משתמשת
DELETE	`/:id`	—	ביטול: המוצר חוזר למכירה, הנקודות מתוקנות	קונה או מנהלת

מועדפים — `/api/favorites`

Method	נתיב	גוף	מחזיר	הרשאה
GET	`/`	—	המוצרים המלאים שסומנו	משתמשת
POST	`/`	productId	המועדף שנוצר	משתמשת
DELETE	`/:productId`	—	הודעת הצלחה	משתמשת

הודעות — `/api/messages`

Method	נתיב	גוף	מחזיר	הרשאה
POST	`/`	name, email, subject, body	אישור קבלה	—

Method	נתיב	גוף	מחזיר	הרשאה
GET	`/`	—	כל ההודעות	מנהלת
PUT	`/:id/read`	—	סימון כנקראה	מנהלת

ניהול — `/api/admin`

Method	נתיב	מחזיר
GET	`/stats`	ארבעת המספרים לכרטיסים העליונים
GET	`/sales-by-month`	Aggregate 1
GET	`/top-sellers`	Aggregate 2
GET	`/users`	כל המשתמשות עם הנקודות שלהן
PUT	`/users/:id/role`	שינוי בין user ל־admin
DELETE	`/users/:id`	מחיקת משתמשת

כל הקבוצה הזו עטופה ב־`router.use(auth, isAdmin)` — שורה אחת שמגנה על כולם. בסך הכול יש כאן יותר מעשרים פניות מהלקוח לשרת, מול דרישה של חמש.

---

8 חיפוש, סינון ומיון

הכול נכנס ל־route אחד, `GET /api/products`, דרך query params. אין endpoint נפרד לחיפוש.

```
GET /api/products?search=שמלה&gender=women&category=clothing&type=dresses
&size=M/38&condition=new&brand=Zara&city=חיפה &minPrice=50&maxPrice=200&sort=price_asc&page=1
```

מסנן	איך זה נראה בסרגל
מגדר	שלושה כפתורים: נשים · גברים · ילדים
קטגוריה	בגדים · נעליים · אביזרים — מופיע אחרי בחירת מגדר
סוג פריט	רשימה שמשתנה לפי הקטגוריה, מתוך `CATEGORIES`
מידה	כפתורי מידה, משתנים לפי הקטגוריה
מצב הפריט	חמש תיבות סימון
מותג	רשימה נפתחת מ־`/api/products/brands`
עיר	רשימת ערים מה־API
טווח מחירים	שני שדות: מ־ עד

ערך `sort`	מיון
`newest`	החדש ביותר (ברירת מחדל)
`price_asc`	מחיר מהנמוך לגבוה
`price_desc`	מחיר מהגבוה לנמוך
`popular`	הכי נצפים

```
const filter = { status: 'available' }; if (search) filter.$or = [ { title: { $regex: search, $options: 'i' } }, { brand: { $regex: search, $options: 'i' } } ]; ['gender','category','type','size','condition','brand','city'] .forEach(f => { if (req.query[f]) filter[f] = req.query[f]; }); if (minPrice || maxPrice) { filter.price = {}; if (minPrice) filter.price.$gte = Number(minPrice); if (maxPrice) filter.price.$lte = Number(maxPrice); } const sortMap = { newest: { createdAt: -1 }, price_asc: { price: 1 }, price_desc: { price: -1 }, popular: { views: -1 } }; const products = await Product.find(filter) .sort(sortMap[sort] || sortMap.newest) .skip((page - 1) * 12).limit(12) .populate('sellerId', 'name city');
```

הלולאה על מערך שמות השדות חוסכת שבע בדיקות `if` כמעט זהות, ואם תוסיפו מסנן — מוסיפים מילה למערך.

---

9 שתי שאילתות Aggregate

מכירות לפי חודש

```
Order.aggregate([ { $match: { status: 'completed' } }, { $group: { _id: { year: { $year: '$createdAt' }, month: { $month: '$createdAt' } }, revenue: { $sum: '$price' }, orders: { $sum: 1 } }}, { $sort: { '_id.year': 1, '_id.month': 1 } } ]);
```

חמש המוכרות המובילות

זו השאילתה שמצדיקה את עצמה כ־Aggregate אמיתי: מקבצת, ממיינת, חותכת ומחברת בין שני Collections. אי אפשר לעשות אותה עם `find`.

```
Order.aggregate([ { $match: { status: 'completed' } }, { $group: { _id: '$sellerId', revenue: { $sum: '$price' }, itemsSold: { $sum: 1 } }}, { $sort: { revenue: -1 } }, { $limit: 5 }, { $lookup: { from: 'users', localField: '_id', foreignField: '_id', as: 'seller' }}, { $unwind: '$seller' }, { $project: { _id: 0, name: '$seller.name', city: '$seller.city', revenue: 1, itemsSold: 1 }} ]);
```

> **למה לא "מוצרים לפי קטגוריה".** זה `$group` על שדה אחד, ומרצה יכולה בצדק לומר שאפשר להשיג את זה עם `find` ולולאה. אם רוצות אותו בגרף — קחו אותו כשלישי, בונוס, לא כאחד משניים.

---

10 שירותים חיצוניים

API ראשון — המרת מטבע

בעמוד המוצר, מתחת למחיר בשקלים, מוצג גם המחיר בדולר. חינמי ובלי מפתח:

```
const res = await fetch('https://open.er-api.com/v6/latest/ILS'); const data = await res.json(); const usd = (price * data.rates.USD).toFixed(0);
```

קוראות לו פעם אחת בטעינת העמוד ושומרות את השער במשתנה, לא בכל כרטיס מוצר בנפרד.

API שני — רשימת ערים בישראל

שדה העיר בהרשמה, בפרסום מוצר ובעמוד התשלום הוא בחירה מרשימה אמיתית של יישובים, לא טקסט חופשי. אם אחת תכתוב "ת״א" ואחרת "תל אביב", הסינון לפי עיר פשוט לא יעבוד.

```
const url = 'https://data.gov.il/api/3/action/datastore_search' + '?resource_id=d4901968-dad3-4845-a9b0-a57d027f11ab&limit=1500'; const res = await fetch(url); const cities = (await res.json()).result.records .map(r => r['שם_ישוב'].trim()).filter(Boolean).sort();
```

> **לפני שמסתמכות על זה.** בדקו שה־API עונה, ושמרו עותק סטטי ב־`public/js/cities.json` כגיבוי. אם השירות נופל ביום ההגנה — המערכת ממשיכה לעבוד. וכדאי לוודא מול המרצה ששניהם נחשבים בעיניה API חיצוני תקין.

תמונות — Cloudinary

הדפדפן מעלה את הקובץ ישירות ל־Cloudinary ומקבל URL, וה־URL הוא מה שנשמר במונגו. אין קבצים בשרת שלכן, ואין תמונות שנעלמות. ההרשמה חינם, ההגדרה רבע שעה — צריך ליצור *unsigned upload preset* בלוח הבקרה.

```
const form = new FormData(); form.append('file', fileInput.files[0]); form.append('upload_preset', 'rewear_unsigned'); const res = await fetch( 'https://api.cloudinary.com/v1_1/YOUR_CLOUD_NAME/image/upload', { method: 'POST', body: form } ); const { secure_url } = await res.json(); // נשמר ב־products.images
```

תוכנית גיבוי אם משהו נתקע: `multer` ששומר ל־`public/uploads`. עובד מצוין על localhost, וזה מה שמריצות בהגנה ממילא. אל תבזבזו על זה יותר מיום.

---

11 נתוני דמה — הקובץ הכי חשוב בפרויקט

סקריפט `seed.js` שמכניס למסד 12 משתמשות, 60 מוצרים בכל המגדרים והקטגוריות, 40 הזמנות מפוזרות על ששת החודשים האחרונים, 25 מועדפים וכמה הודעות.

יש לו שתי מטרות, והשנייה חשובה יותר מהראשונה.

הראשונה, מראה: בלי נתונים הגרפים יציגו עמודה אחת, החנות תיראה ריקה, והמיון לפי מחיר לא יוכיח כלום. עם נתונים, המערכת נראית אמיתית.

השנייה, והיא הקריטית — עצמאות. ברגע שיש נתונים במסד, מי שבונה את החנות לא מחכה שמישהי תסיים את טופס פרסום המוצר. מי שבונה את הדשבורד לא מחכה שמישהי תסיים הזמנות. כל אחת מפתחת מול נתונים שכבר קיימים. בגלל זה הסקריפט הזה נבנה ב**שבוע 0, לפני כל השאר** — הוא מה שמאפשר לחמשתכן לעבוד במקביל.

```
"scripts": { "dev": "nodemon server.js", "seed": "node seed.js" }
```

הסקריפט מוחק את כל ה־Collections ובונה מחדש, כדי שכל אחת תוכל לאפס את הבסיס שלה מתי שתרצה. שתי משתמשות קבועות תמיד נוצרות, ואת אלה כולכן מכירות בעל פה:

```
admin@rewear.co.il / Admin123 → מנהלת noa@rewear.co.il / User123 → 500 נקודות, משתמשת רגילה
```

---

12 מדריך התחלה — שבוע 0

חמישה ימים שכולכן יושבות על אותו קוד. אל תתפצלו לפני שהחלק הזה עומד — כל אחת תיגע בבסיס הזה מאה פעם, ואם הוא לא זהה אצל כולן, שום דבר לא יתחבר.

יום 1 — חשבונות וסביבה

כל אחת מתקינה על המחשב שלה: Node.js (גרסה 18 ומעלה), VS Code, Git ו־MongoDB Compass. פותחות חשבון MongoDB Atlas אחד משותף לצוות, יוצרות Cluster חינמי, ומוסיפות ב־Network Access את הכתובת `0.0.0.0/0` כדי שכולכן תוכלו להתחבר מכל מקום. פותחות חשבון Cloudinary אחד ויוצרות unsigned upload preset בשם `rewear_unsigned`.

מפתחת 1 יוצרת Repository ב־GitHub ומוסיפה את כולן כ־Collaborators.

יום 2 — השלד עולה

```
mkdir rewear && cd rewear npm init -y npm install express mongoose bcryptjs jsonwebtoken dotenv npm install --save-dev nodemon
```

יוצרות `.gitignore` עם שתי שורות — `node_modules` ו־`.env` — ואת `.env.example` שכן נכנס ל־Git:

```
PORT=3000 MONGO_URI=your_connection_string_here JWT_SECRET=your_secret_here
CLOUDINARY_CLOUD_NAME=your_cloud_name
```

`server.js` המינימלי:

```
require('dotenv').config(); const express = require('express'); const mongoose = require('mongoose'); const path = require('path'); const app = express(); app.use(express.json()); app.use(express.static(path.join(__dirname, 'public'))); mongoose.connect(process.env.MONGO_URI) .then(() => console.log('MongoDB connected')) .catch(err => console.error('MongoDB error:', err.message)); app.get('/api/health', (req, res) => res.json({ ok: true })); const PORT = process.env.PORT || 3000; app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```

מריצות `npm run dev`, פותחות `localhost:3000/api/health`, ורואות `{"ok":true}`.
עד שזה לא עובד אצל כל החמש, לא ממשיכות.

יום 3 — המודלים וה־middleware

כותבות את חמשת המודלים לפי סעיף 4, מילה במילה. אחר כך את `middleware/auth.js`:

```
const jwt = require('jsonwebtoken'); function auth(req, res, next) { const token = req.headers.authorization?.split(' ')[1]; if (!token) return res.status(401).json({ message: 'נדרשת התחברות' }); try { req.user = jwt.verify(token, process.env.JWT_SECRET); // { id, role } next(); } catch { res.status(401).json({ message: 'ההתחברות פגה, יש להתחבר מחדש' }); } } function isAdmin(req, res, next) { if (req.user.role !== 'admin') return res.status(403).json({ message: 'אין לך הרשאה לפעולה הזו' }); next(); } module.exports = { auth, isAdmin };
```

בעלות על מוצר נבדקת בתוך ה־controller, כי היא תלויה בנתון מהמסד:

```
if (product.sellerId.toString() !== req.user.id && req.user.role !== 'admin') return res.status(403).json({ message: 'אפשר לערוך רק מוצרים שלך' });
```

יום 4 — הנתונים נכנסים

כותבות את `seed.js` ומריצות `npm run seed`. פותחות Compass ורואות את חמשת ה־Collections מלאים. זה הרגע שבו הפרויקט מפסיק להיות תיאורטי.

גם: `public/js/categories.js` עם עץ הקטגוריות והמידות מסעיף 5.

יום 5 — הבסיס של הצד הלקוח

`public/js/common.js` עם ארבע פונקציות שכולן ישתמשו בהן:

```
function getUser() { return JSON.parse(localStorage.getItem('rewear_user') || 'null'); } function logout() { localStorage.clear(); location.href = '/index.html'; } function renderNavbar() { /* ממלא #navbar לפי getUser() */ } async function api(path, options) { /* סעיף 2 */ }
```

`public/css/style.css` עם המשתנים, הפונטים, הכפתורים, שדות הטופס וכרטיס המוצר.
וקובץ HTML אחד לדוגמה שמראה איך כל עמוד נראה: את ה־`<div id="navbar">`, את הטעינה של הסקריפטים ואת סדר הקבצים.

Commit ותג `v0.1`. עכשיו מתפצלות.

---

13 חלוקת העבודה בין חמשתכן

החלוקה היא **לפי פיצ'ר, לא לפי שכבה**. כל אחת מקבלת יכולת שלמה מהמסד ועד המסך: המודל, ה־routes, ה־controller, העמוד וה־JS שלו. בחלוקה לפי שכבות כל אחת חסומה על ידי מישהי אחרת; בחלוקה לפי פיצ'ר אף אחת לא מחכה, וכל אחת יודעת להסביר בהגנה חלק שלם.

מפתחת 1 — הזדהות ומשתמשות

בעלת האינטגרציה של הצוות

- הרשמה והתחברות מקצה לקצה, bcrypt והנפקת JWT
- `login.html` עם שני הטאבים ולידציה בצד לקוח
- טאב "הפרטים שלי" וטאב "הנקודות שלי" בפרופיל
- ניהול משתמשות במסך המנהלת: טבלה, שינוי תפקיד, מחיקה
- בעלת ה־Repository: מאשרת merge, פותרת קונפליקטים, שומרת שהענף הראשי תמיד רץ

> **מתחילה מ:** `POST /api/auth/register` בלבד, ובודקת אותו ב־Postman עד שהוא מחזיר טוקן. אחר כך login, ורק אז נוגעת ב־HTML.

קבצים: `authRoutes.js` · `authController.js` · `login.html` · `js/auth.js` · חלק המשתמשות ב־ `adminRoutes.js`

מפתחת 2 — פרסום וניהול מוצרים

ה־CRUD המרכזי של הפרויקט

- ארבע פעולות ה־CRUD המלאות על מוצרים, כולל בדיקת בעלות
- טופס פרסום מוצר עם התלות בין מגדר, קטגוריה, סוג ומידה
- אותו טופס במצב עריכה, טעון בנתוני המוצר
- העלאת תמונות ל־Cloudinary עם תצוגה מקדימה, עד ארבע תמונות
- טאב "המוצרים שלי", מחיקה עם אישור, תגית "נמכר"

> **מתחילה מ:** `GET /api/products` ו־`POST /api/products` ב־Postman. אחר כך הטופס — קודם השדות הפשוטים, ורק בסוף התמונות והתפריטים המקושרים.

קבצים: `productRoutes.js` · `productController.js` · `js/products.js` · `js/upload.js` · חלק המוצרים ב־`profile.html`

מפתחת 3 — חנות, סינון ועמוד מוצר

מה שהמשתמשת רואה ראשון

- `shop.html` : כל המוצרים בעמוד אחד, סרגל סינון מלא, מיון, חיפוש, דפדוף
- לוגיקת הסינון והמיון בשרת לפי סעיף 8
- `product.html` : גלריית תמונות, כל הפרטים, פרטי המוכרת, מונה צפיות
- `index.html` : באנר, כפתורי מגדר, הפריטים האחרונים
- API המרת המטבע

> **מתחילה מ:** רשת המוצרים בלי שום מסנן — `GET /api/products` ומציגה כרטיסים. מסננים מוסיפים אחד־אחד אחרי שהרשת עובדת, לא לפני.

קבצים: `shop.html` · `product.html` · `index.html` · `js/shop.js` · `js/product.js` · `js/currency.js`

מפתחת 4 — תשלום, נקודות, הזמנות ומועדפים

החלק שהכי מרשים בהגנה

- `checkout.html` : סיכום הפריט, בחירה בין אשראי לנקודות, טופס כרטיס עם Luhn
- זרימת הרכישה בשרת: תפיסת המוצר, חיוב, זיכוי המוכרת, יצירת ההזמנה
- טאב "ההזמנות והמכירות שלי", כולל ביטול
- מועדפים: כפתור הלב בכל כרטיס ובעמוד המוצר, וטאב המועדפים
- API הערים, כולל קובץ הגיבוי

> **מתחילה מ:** `POST /api/orders` ב־Postman עם תשלום בנקודות בלבד — הכי פשוט. אחרי שהיתרות זזות נכון במסד, מוסיפה את מסך התשלום ואת האשראי.

קבצים: `checkout.html` · `orderRoutes.js` · `favoriteRoutes.js` + controllers · `js/checkout.js` · `js/orders.js` · `js/favorites.js`

מפתחת 5 — ניהול, עמודי תוכן ועיצוב

שתי ה־Aggregate והמראה הכללי

- שתי שאילתות ה־Aggregate וה־endpoints שלהן
- ארבעת כרטיסי המספרים ושני הגרפים ב־Chart.js
- טבלת ניהול המוצרים ותיבת ההודעות במסך המנהלת
- שלושת עמודי התוכן: `how-it-works.html` , `about.html` , `contact.html` וטופס יצירת הקשר
- אחריות על `style.css` : אחידות בין עשרת העמודים, RTL, התאמה למסך צר, מסכי ריק והודעות שגיאה

> **מתחילה מ:** שתי ה־Aggregate ב־Compass, בלשונית Aggregations — שם רואים את התוצאה מיד ובלי לכתוב שרת. רק כשהצינור נכון מעתיקה אותו ל־controller.

קבצים: `admin.html` · `how-it-works.html` · `about.html` · `contact.html` · `js/admin.js` · `js/contact.js` · `messageRoutes.js` · `css/style.css`

המפגש השבועי. פעם בשבוע, חצי שעה, כל אחת מדגימה על המסך שלה מה עובד. לא מדווחת — מדגימה. זה תופס פערים בזמן שאפשר עוד לתקן אותם.

14 מי יכולה לעבוד לבד ומי מחכה

התשובה הקצרה: אחרי שבוע 0, כל החמש עובדות במקביל ואף אחת לא חסומה. זה לא במקרה — זה בדיוק מה ששבוע 0 נועד להשיג.

למה אף אחת לא מחכה

שלושה דברים משחררים את התלויות, וכולם נמצאים בשבוע 0:

- **נתוני הדמה** — יש מוצרים, משתמשות והזמנות במסד מהיום הראשון. מי שבונה חנות לא מחכה למי שבונה טופס פרסום. מי שבונה דשבורד לא מחכה למי שבונה הזמנות.
- **ה־middleware** — `auth` ו־`isAdmin` קיימים לפני שההרשמה נכתבה, אז כל אחת יכולה להגן על ה־routes שלה מיד.
- **המסמך הזה** — כל ה־routes וכל שמות השדות מוגדרים מראש, אז אפשר לכתוב `fetch` לנתיב שעדיין לא קיים ולדעת שהוא יעבוד כשהוא ייכתב.

איך משיגים טוקן לפני שההרשמה מוכנה

זו התלות היחידה שנשארת: מי שבונה פיצ'ר שדורש התחברות צריכה טוקן כדי לבדוק אותו. הפתרון קטן — מפתחת 1 מייצרת שני טוקנים ידניים ושולחת בקבוצה ביום הראשון:

```
// scripts/maketoken.js require('dotenv').config(); const jwt = require('jsonwebtoken'); console.log(jwt.sign({ id: 'ID_מהסיד', role: 'user' }, process.env.JWT_SECRET)); console.log(jwt.sign({ id: 'ID_מהסיד', role: 'admin' }, process.env.JWT_SECRET));
```

מדביקות את הטוקן ב־Postman או ישירות ב־`localStorage` של הדפדפן, וממשיכות לעבוד. כשההרשמה האמיתית מוכנה, שום דבר לא צריך להשתנות — זה אותו טוקן בדיוק.

מי	יכולה להתחיל	תלויה במישהי?	מתי חייבת לחכות
מפתחת 1	מיד	לא	אף פעם
מפתחת 2	מיד — יש מוצרים בסיד	לא	רק לשילוב סופי של ההעלאה מהטופס האמיתי
מפתחת 3	מיד — 60 מוצרים בסיד	לא	אף פעם
מפתחת 4	מיד — יש משתמשות ומוצרים	לא	קישור מעמוד המוצר לתשלום, בשבוע 3
מפתחת 5	מיד — יש 40 הזמנות בסיד	לא	אף פעם

שלוש נקודות מפגש

אלה הרגעים היחידים שבהם שתי בנות חייבות לדבר. שווה לסמן אותם ביומן:

1. **סוף שבוע 1 — 1 ← 3, 4:** ההתחברות האמיתית מחליפה את הטוקן הידני. כולן מוחקות את `localStorage` ומתחברות דרך המסך.
2. **אמצע שבוע 3 — 3 ← 4:** כפתור הרכישה בעמוד המוצר מפנה ל־`checkout.html?id=` . שורה אחת של קוד, אבל צריך לתאם מתי.
3. **סוף שבוע 3 — 2 ← 3:** מוצר שנוצר בטופס האמיתי מופיע בחנות עם כל השדות. כאן מתגלות אי־התאמות בשמות שדות, אם נשארו כאלה.

סדר עבודה בתוך היום

לכל אחת, לכל פיצ'ר, אותם ארבעה שלבים. אל תדלגו על הראשון:

1. **Postman.** כותבות את ה־route ובודקות אותו לבד, בלי HTML. רואות שהוא מחזיר את מה שכתוב במסמך.
2. **Compass.** בודקות שהנתון באמת נכנס או השתנה במסד.
3. **HTML.** בונות את המסך עם נתונים קשיחים, בלי fetch, רק כדי לראות שהעיצוב עומד.
4. **חיבור.** מחליפות את הנתונים הקשיחים ב־`api()` .

מי שמתחילה מ־HTML מגלה באגים בשלושה מקומות בבת אחת ולא יודעת איפה הבעיה. מי שמתחילה מ־Postman יודעת תמיד בדיוק איפה נשבר.

---

15 מבנה התיקיות

```
rewear/
├── server.js
├── seed.js
├── .env (לא נכנס ל־git)
├── .env.example
├── .gitignore
├── package.json
│
├── config/db.js
├── middleware/auth.js
│
├── models/
│   └── User.js Product.js Order.js Favorite.js Message.js
│
├── routes/
│   └── authRoutes.js productRoutes.js orderRoutes.js
│   └── favoriteRoutes.js messageRoutes.js adminRoutes.js
│
├── controllers/ (קובץ מקביל לכל routes)
│
└── public/
    ├── index.html shop.html product.html login.html
    ├── checkout.html profile.html admin.html
    ├── how-it-works.html about.html contact.html
    │
    ├── css/style.css
    │
    └── js/
        ├── common.js categories.js cities.json
        ├── auth.js products.js upload.js
        ├── shop.js product.js currency.js
        ├── checkout.js orders.js favorites.js
        ├── admin.js contact.js
```

הפרדה מלאה בין HTML, CSS ו־JS. אין `<style>` ואין `<script>` עם קוד בתוך קובצי ה־HTML.

---

16 עבודה עם Git

- ענף `main` תמיד רץ. אף אחת לא דוחפת אליו ישירות.
- כל אחת בענף משלה: `feature/auth` , `feature/products` , `feature/shop` , `feature/checkout` , `feature/admin` .

- Pull Request למיזוג. מפתחת 1 בודקת שהפרויקט עדיין עולה, ואז ממזגת.
- Commit כל יום עבודה. הודעות ברורות: `"add product edit form"`, לא `"fix"`.
- לפני שמתחילות לעבוד בבוקר: `git pull origin main`. תמיד.

המרצה מבקשת שלוש גרסאות בתאריכים שונים. תייגו תשע, כדי שיהיה ברור שהמערכת נבנתה לאורך שישה שבועות ולא בשלושה לילות:

תג	מה נכנס
`v0.1`	שלד, מודלים, middleware, נתוני דמה
`v0.2`	הרשמה והתחברות
`v0.3`	CRUD מלא על מוצרים
`v0.4`	חנות עם כל המסננים
`v0.5`	תשלום, נקודות והזמנות
`v0.6`	מועדפים ועמוד מוצר מלא
`v0.7`	מסך ניהול ו־Aggregate
`v0.8`	עמודי תוכן ושני ה־API
`v1.0`	גרסת ההגשה

17 לוח זמנים עד 19.10

13.9–7	שבוע 0 לפי סעיף 12. כולן ביחד. בסופו: השרת עולה, המסד מלא, ה־middleware עובד, יש עיצוב בסיסי.
20.9–14	מתפצלות. הרשמה והתחברות אמיתיות; CRUD מוצרים; רשת המוצרים בחנות; רכישה בנקודות ב־Postman; שתי ה־Aggregate.
27.9–21	טופס פרסום עם תמונות; כל המסננים; עמוד תשלום עם אשראי; מועדפים; דשבורד עם גרפים.
4.10–28.9	עמוד מוצר מלא; פרופיל על כל הטאבים; ניהול משתמשות והודעות; שלושת עמודי התוכן; שני ה־API.
11.10–5	שילוב מלא. שלוש נקודות המפגש מסעיף 14 נסגרות. עיצוב אחיד, מסכי ריק, התאמה למסך צר, תיקון באגים.
18.10–12	**הקפאת פיצ'רים.** לא מוסיפות שום דבר חדש. רק בדיקות, תיקונים, וחזרה על תרחיש ההגנה עד שהוא רץ חלק.

19.10	הגשה.
20.10	הגנה.

> **ההקפאה בשבוע האחרון היא לא המלצה.** פיצ'ר שנוסף שלושה ימים לפני ההגשה שובר דברים שכבר עבדו, ואין זמן לגלות את זה. מערכת פשוטה שרצה בלי תקלה עדיפה בהרבה על מערכת עשירה שנתקעת מול המרצה.

18 מתי פיצ'ר נחשב גמור

לפני שמישהי אומרת "סיימתי", כל השורות האלה נכונות:

- עובד בדפדפן מההתחלה עד הסוף, בלי לפתוח את הקונסולה
- הקונסולה נקייה משגיאות
- הנתון נכנס ל־MongoDB, ונראה בעיניים ב־Compass
- נבדק כמשתמשת מחוברת, כאורחת וכמנהלת
- שגיאה מהשרת מוצגת למשתמשת כהודעה בעברית, לא נבלעת בשקט
- מצב ריק מטופל — מה רואים כשאין מוצרים ברשימה
- נבדק גם במסך צר של טלפון
- הקוד ממוזג ל־`main` וה־`main` עדיין רץ אצל כולן

19 תרחיש ההגנה

זה מה שצריך לרוץ חלק, בלי היסוס ובלי לחפש טאבים באמצע. תרגלו אותו מההתחלה ועד הסוף לפחות שלוש פעמים בשבוע האחרון, כשכל אחת מדברת על החלק שלה:

1. הרשמה של משתמשת חדשה
2. Compass — המשתמשת נמצאת ב־`users`, הסיסמה מוצפנת, הנקודות 0
3. התחברות
4. פרסום בגד: נשים ← בגדים ← שמלות, מידה, מצב, מותג ותמונה אמיתית
5. Compass — המוצר נמצא עם `sellerId` נכון
6. חנות — סינון לפי מגדר, אחר כך לפי סוג פריט, אחר כך מיון לפי מחיר
7. חיפוש חופשי לפי שם או מותג
8. עריכת המוצר, שינוי המחיר, והמחיר החדש מופיע בחנות
9. התחברות ממשתמשת שנייה, הוספה למועדפים, ומעבר לעמוד התשלום
10. תשלום באשראי: הקלדת מספר לא תקין ← נפסל. מספר תקין ← עובר
11. המוצר סומן כנמכר ויצא מהחנות
12. המוכרת מתחברת ורואה שהנקודות שלה עלו
13. המוכרת קונה פריט אחר *בנקודות* שצברה
14. ניסיון להיכנס ל־`admin.html` כמשתמשת רגילה — נחסם
15. התחברות כמנהלת: ארבעת הכרטיסים, שני הגרפים, טבלת המשתמשות, תיבת ההודעות
16. הצגת קוד שתי שאילתות ה־Aggregate והסבר מה כל שלב עושה
17. הצגת שני ה־API: המחיר בדולר, ורשימת הערים
18. היסטוריית ה־Git עם התגים לאורך שישה שבועות

לפני שנכנסות. הריצו `npm run seed` על מסד נקי, פתחו מראש את Compass, את הדפדפן ואת ה־Git, וודאו ששתי משתמשות בדיקה מוכנות בשני חלונות נפרדים.

---

20 מה לא נכנס לפרויקט

הרשימה הזאת שווה לא פחות מכל השאר. כל אחד מהדברים האלה שואב ימי עבודה ולא מוסיף כמעט נקודה מול הדרישות:

- סליקה אמיתית, Stripe או PayPal
- משיכת נקודות לכסף — זה מחזיר את כל בעיית הסליקה מהדלת האחורית
- עמלת פלטפורמה או הנחות
- עגלת קניות עם כמה פריטים — כל פריט הוא יחיד, אז רכישה ישירה מספיקה
- צ'אט בזמן אמת בין קונה למוכרת

- מערכת משלוחים ומעקב חבילות
- התראות במייל או ב־SMS
- המלצות מבוססות AI

אם יישאר זמן אחרי שהכול עובד, אלה התוספות שנותנות הכי הרבה במעט מאמץ: תמונת פרופיל, "פריטים דומים" בתחתית עמוד המוצר, פילטר צבע, ומצב לילה.

ReWear · מסמך אפיון גרסה 2.0 · כל שינוי — מעדכנים כאן קודם ומודיעים בקבוצה.